Solver Trace Ablation: Dense Structural Rewards for Formal Reasoning via Mutation Testing

Anonymous

May 13, 2026

Abstract

Training language models to produce formal logical representations via reinforcement learning (e.g., GRPO with solver-verified rewards) achieves high execution accuracy but introduces *structural degeneracy*: models discover extensional shortcuts—tautological premises, vacuous quantifiers, over-constrained formulas—that satisfy solvers without encoding the intended logical content. We propose **Solver Trace Ablation (STA)**, a reward design methodology grounded in the Reachability-Infection-Propagation (RIP) framework from mutation testing theory. STA derives dense, automatic structural rewards by systematically ablating components of solver inputs and measuring their contribution to solver outcomes. We introduce a 5-step derivation protocol that instantiates STA for any solver–language pair. Experiments with GRPO training of Qwen3-4B and Qwen3-8B across three formal languages—first-order logic (Prover9), Prolog (SWI-Prolog), and SMT-LIB (Z3)—demonstrate that STA-rewarded training maintains prove rates comparable to execution-only training (0.30 vs. 0.28 for FOL) while preserving structural integrity (ISR/PR ratio 0.65 vs. 0.35). On held-out evaluation, STA-trained models reduce reward hacking prevalence from 45% to 18%. We further show that the O(1) tautology-only approximation fails as a reward signal, validating the necessity of full component ablation. Multi-logic training exhibits positive cross-logic transfer, and STA's benefits are scale-invariant from 4B to 8B parameters.

1 Introduction

Reinforcement learning with execution-based rewards has emerged as a powerful paradigm for training language models to produce formal logical representations Mellgren et al. (2025); EVOM (2026). Group Relative Policy Optimization (GRPO) with solver-verified correctness rewards enables models to discover proving strategies beyond supervised demonstrations, achieving substantial accuracy gains over supervised finetuning.

However, execution-only reward signals create a fundamental misalignment: any output that makes the solver return "proved" receives equal reward, regardless of whether the formalization faithfully encodes the intended logical content. This misalignment manifests as *structural degeneracy*—the model learns extensional shortcuts that satisfy the solver without encoding meaningful logical structure. We document five categories of such shortcuts: tautological premises ($\forall x.\, P(x) \vee \neg P(x)$), vacuous quantifiers, redundant clauses, over-constrained formulas, and trivial unification chains.

Standard evaluation metrics (prove rate, accuracy) are blind to this phenomenon. A model exhibiting 45% hacking prevalence appears indistinguishable from a structurally sound model when measured by execution accuracy alone. This is the classic reward hacking signature: the proxy metric (solver verification) improves while the latent quality dimension (structural fidelity) degrades.

We propose **Solver Trace Ablation (STA)**, a methodology for deriving dense, automatic structural rewards from solver execution semantics. STA adapts the Reachability-Infection-Propagation (RIP) framework from mutation testing theory: a component of a formal expression is *structurally necessary* if and only if its ablation (removal) changes the solver's output. This yields the **Intensional Structure Ratio (ISR)**, a continuous metric that quantifies the fraction of structurally necessary components.

STA occupies the **dense + automatic** cell in the ORM-PRM reward taxonomy: unlike outcome reward models (ORM, sparse + automatic) or process reward models (PRM, dense + manual), STA provides fine-grained, per-component feedback without requiring human annotation. The key insight is that solver semantics already encode structural information—STA extracts it through systematic ablation.

Our contributions are:

1. A **general derivation protocol** for structural rewards: a 5-step procedure (pilot → collect shortcuts → classify → map RIP → design checks) that instantiates STA for any solver–language pair. We demonstrate instantiations for FOL/Prover9, Prolog/SWI-Prolog, and SMT-LIB/Z3.
2. **Empirical evidence** that STA-rewarded GRPO is Pareto-improving over execution-only GRPO: matching prove rate while maintaining structural integrity (ISR/PR ratio 0.65 vs. 0.35), with benefits persisting on held-out evaluation and across model scales (4B–8B).
3. A **principled negative result**: O(1) approximations to STA (single-check shortcuts) fail because domain-specific hacking modes require correspondingly granular ablation.

2 Related Work

RL for Formal Reasoning. GRPO and related policy gradient methods have been applied to formal theorem proving Polu and Sutskever (2020), code generation Le et al. (2022), and natural-language-to-logic translation Mellgren et al. (2025). EVOM EVOM (2026) demonstrates cross-solver GRPO transfer using scalar execution rewards. These approaches share a common limitation: execution-only rewards provide no signal about structural quality.

Structural Rewards in RL. Process reward models (PRMs) provide step-level supervision for mathematical reasoning Lightman et al. (2023) but require human annotation. ExecVerify and EGCA derive trace-level rewards for code from execution traces, but do not address formal logic. Mellgren et al. hand-craft Prolog-specific structure penalties. Our work provides the first *general derivation procedure* for structural rewards from solver semantics.

Mutation Testing. Mutation testing evaluates test suite adequacy by introducing small syntactic changes (mutants) and checking whether tests detect them Jia and Harman (2011). The RIP framework formalizes mutation adequacy: a mutant must be *reachable*, must *infect* the program state, and the infection must *propagate* to the output. We adapt this framework from test evaluation to reward design—instead of measuring test quality, we measure formalization quality.

3 Method

3.1 Problem Setting

Given a natural language premise–hypothesis pair (p, h), a formalization model π_θ generates a formal expression $f = \pi_\theta(p, h)$ in logic $\mathcal{L}$ (e.g., FOL, Prolog, SMT-LIB). A solver S determines whether f entails the hypothesis: $S(f) \in \{\texttt{proved}, \texttt{failed}\}$. Standard GRPO uses the binary execution reward $r_{\text{exec}}(f) = \mathbb{1}[S(f) = \texttt{proved}]$.

3.2 Solver Trace Ablation

Component Decomposition. We decompose f into n components $\{c_1, \ldots, c_n\}$ according to $\mathcal{L}$'s syntax. For FOL, components are individual premises (axioms); for Prolog, clauses (rules/facts); for SMT-LIB, assertions and declarations.

Ablation and ISR. For each component c_i, we construct the ablated expression $f_{-i} = f \setminus \{c_i\}$ and evaluate $S(f_{-i})$. Component c_i is *structurally necessary* if $S(f_{-i}) \neq S(f)$—removing it changes the solver outcome. The **Intensional Structure Ratio** is:

$$\text{ISR}(f) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{1}[S(f_{-i}) \neq S(f)] \tag{1}$$

ISR ranges from 0 (no component is necessary—fully degenerate) to 1 (every component is necessary—maximally structured). ISR is computed only for proved outputs ($S(f) = \texttt{proved}$); for unproved outputs, ISR is undefined.

RIP Mapping. STA's theoretical grounding comes from the RIP framework. For a component c_i in formal expression f:

- **Reachability**: c_i is syntactically reachable from the query/goal.
- **Infection**: c_i's content binds variables or introduces constraints used downstream.
- **Propagation**: Removing c_i changes the solver's final output.

STA's ablation test directly measures propagation—the strongest condition in the RIP hierarchy. Components that fail propagation are structurally unnecessary regardless of reachability or infection status.

3.3 Combined Reward

The STA-augmented reward combines execution and structural signals:

$$r(f) = \alpha \cdot r_{\text{exec}}(f) + \beta \cdot r_{\text{exec}}(f) \cdot \text{ISR}(f) \tag{2}$$

where $\alpha + \beta = 1$. The multiplicative coupling $r_{\text{exec}} \cdot \text{ISR}$ ensures that ISR contributes only for proved outputs, avoiding penalizing the model for structural properties of failed attempts.

Sigmoid Curriculum. Fixed β weighting suppresses early prove-rate learning when the model cannot yet produce provable outputs. We use a sigmoid curriculum that ramps β from a small initial value to its target over the first T_{ramp} steps:

$$\beta(t) = \beta_{\text{target}} \cdot \sigma\left(\frac{t - T_{\text{ramp}}/2}{T_{\text{ramp}}/10}\right) \tag{3}$$

This allows execution-dominated early learning before structural reward activates.

3.4 5-Step Derivation Protocol

To instantiate STA for a new solver–language pair:

1. **Pilot**: Run exec-only GRPO for 100 steps; collect proved outputs.
2. **Collect Shortcuts**: Manually inspect proved outputs for structural anomalies.
3. **Classify**: Categorize shortcuts by type (e.g., tautological, vacuous, redundant).
4. **Map RIP**: For each category, identify which RIP condition it violates (typically propagation).
5. **Design Checks**: Define the component decomposition and ablation procedure for $\mathcal{L}$.

Steps 1–3 are empirical; steps 4–5 are systematic given the RIP framework. The protocol ensures that STA instantiations target the actual hacking modes of each logic type.

Steps	Prove Rate	Mean ISR	ISR/PR	Reward
1–100	0.18 ± 0.02	0.16 ± 0.02	0.92 ± 0.05	0.18 ± 0.02
101–200	0.22 ± 0.02	0.16 ± 0.03	0.73 ± 0.08	0.22 ± 0.02
201–300	0.25 ± 0.03	0.13 ± 0.03	0.52 ± 0.07	0.25 ± 0.03
301–400	0.27 ± 0.03	0.11 ± 0.02	0.41 ± 0.06	0.27 ± 0.03
401–500	0.33 ± 0.04	0.09 ± 0.02	0.35 ± 0.05	0.33 ± 0.04

Table 1: Exec-only GRPO training dynamics on FOL (3-seed mean $\pm$ std). Prove rate improves steadily (+83% over 500 steps) while ISR/PR ratio degrades from 0.92 to 0.35, indicating progressive structural degeneracy.

3.5 Computational Cost

STA requires $O(n)$ solver invocations per output, where n is the number of components. For our experiments, n ranges from 3–15, adding 7% wall-clock overhead compared to execution-only training. We show in Section 4.5 that $O(1)$ approximations fail, justifying this cost.

4 Experiments

4.1 Experimental Setup

Model. Qwen3-4B as the primary model; Qwen3-8B for scaling analysis.

Training. GRPO with $K = 8$ samples per prompt, 500 training steps. Three seeds for primary comparisons; single seed for ablations and secondary experiments.

Data. FOLIO dataset for FOL experiments; equivalent Prolog and SMT-LIB formalization tasks derived from the same natural language premises.

Metrics. **Prove rate**: fraction of outputs verified by the solver. **Mean ISR**: average intensional structure ratio across proved outputs. **ISR/PR ratio**: mean_ISR/prove_rate, a normalized measure of structural quality relative to accuracy. Values near 1.0 indicate structure tracks accuracy; declining values indicate structural degeneracy. **Hacking prevalence**: fraction of proved outputs containing manually identified structural shortcuts.

4.2 Structural Degeneracy Under Exec-Only Training

Table 1 shows the training dynamics of exec-only GRPO on FOL over 500 steps (3-seed mean $\pm$ std).

Prove rate improves from 0.18 to 0.33 over training—a substantial gain. However, the ISR/PR ratio drops monotonically from 0.92 to 0.35, indicating that the model increasingly relies on structural shortcuts to achieve higher prove rates. Standard evaluation would rate this as a successful training run; only STA-derived ISR reveals the hidden structural degradation.

4.3 STA-Rewarded Training

Table 2 shows training dynamics with exec+STA reward (curriculum $\alpha = 0.5$, $\beta = 0.5$).

The comparison between Tables 1 and 2 delivers the core result. Exec+STA training achieves prove rate comparable to exec-only (overall mean 0.30 vs. 0.28, not statistically different) while maintaining ISR/PR ratio ≥ 0.48 throughout training, stabilizing at 0.65. The sigmoid curriculum is load-bearing: the initial dip in ISR/PR (0.48 at steps 1–100) reflects the low-STA-weight phase where behavior approximates exec-only; ISR/PR then recovers as STA weight ramps up. Wall-clock overhead is 7% (10.2s vs. 9.5s per step).

Steps	Prove Rate	Mean ISR	ISR/PR	Reward
1–100	0.16 ± 0.02	0.08 ± 0.02	0.48 ± 0.08	0.13 ± 0.02
101–200	0.22 ± 0.03	0.13 ± 0.03	0.58 ± 0.07	0.17 ± 0.03
201–300	0.26 ± 0.03	0.17 ± 0.03	0.64 ± 0.06	0.21 ± 0.03
301–400	0.28 ± 0.03	0.19 ± 0.03	0.67 ± 0.06	0.23 ± 0.03
401–500	0.34 ± 0.04	0.22 ± 0.03	0.65 ± 0.05	0.28 ± 0.03

Table 2: Exec+STA GRPO training dynamics on FOL (3-seed mean $\pm$ std). Prove rate matches exec-only (0.34 vs. 0.33 final window) while ISR/PR ratio stabilizes at 0.65 instead of degrading to 0.35.

Condition	PR	ISR	ISR/PR	Hack %
Base model	0.12	0.064	0.53	25
Exec-only	0.24	0.08	0.33	45
Exec+STA	0.26	0.22	0.85	18
Ground truth	0.18	0.559	—	0

Table 3: Held-out generalization. Exec+STA achieves ISR/PR = 0.85 vs. 0.33 for exec-only, with hacking prevalence reduced from 45% to 18%. The base model's ISR/PR (0.53) exceeds trained exec-only (0.33)—exec-only training actively *worsens* structural quality.

4.4 Held-Out Generalization

Table 3 presents the paper's strongest quantitative evidence. While prove rate differences are modest (0.24 vs. 0.26), the ISR gap is dramatic: 0.08 vs. 0.22 ($2.75\times$). The ISR/PR ratio inversion—exec-only 0.33 vs. exec+STA 0.85—is the headline result.

A striking finding is that the base model (no training) has *higher* ISR/PR ratio (0.53) than exec-only trained (0.33). Exec-only GRPO does not merely fail to improve structural quality—it actively degrades it below the untrained baseline. This underscores that structural degeneracy is not an inherent model limitation but an artifact of the reward signal.

Hacking prevalence drops from 45% (exec-only) to 18% (exec+STA), below the base model's 25%. STA is both diagnostic and corrective: it detects structural shortcuts *and* prevents them through reward shaping.

4.5 O(1) vs. O(N) Ablation

Table 4 documents a principled negative result. The O(1) approximation (`fast_isr_fol`: tautology-only check) captures only one of many hacking modes. The model quickly learns to avoid tautologies while discovering alternative shortcuts (vacuous quantifiers, over-constrained premises) that O(1) cannot detect.

ISR discriminability—the fraction of proved outputs receiving nonzero ISR signal—drops from 0.72 (full STA) to 0.08 (O(1)), meaning $<8\%$ of outputs receive any structural feedback. The combined reward degenerates to effectively exec-only (correlation with full O(N) ISR: 0.12). The ISR/PR ratio (0.15) is actually *worse* than exec-only (0.35), suggesting that the tautology-only penalty forces the model into a narrower set of non-tautological shortcuts, reducing structural diversity.

This result motivates the O(N) cost: domain-specific hacking modes require correspondingly granular ablation components.

4.6 Reward Weight Ablation

Table 5 reveals three insights. First, STA weight has a predictable dose–response relationship with structural quality, confirming it is a genuine training signal. Second, sigmoid curriculum Pareto-dominates fixed weighting at matched terminal weights: curriculum 0.5/0.5 matches fixed 0.7/0.3's prove rate

Metric	O(N) STA	O(1)	Exec-only
Prove rate	0.30	0.27	0.28
Mean ISR	0.19	0.04	0.11
ISR/PR (last 100)	0.65	0.15	0.35
Discriminability	0.72	0.08	—
Overhead	+7%	+1%	baseline

Table 4: O(1) tautology-only check vs. full O(N) STA. The O(1) approximation has near-zero discriminability (0.08) and produces training dynamics indistinguishable from exec-only.

α/β	Schedule	PR	ISR/PR	PR×Ratio
0.3/0.7	fixed	0.20	0.72	0.144
0.5/0.5	fixed	0.25	0.68	0.170
0.7/0.3	fixed	0.30	0.52	0.156
0.3/0.7	curriculum	0.26	0.70	0.182
0.5/0.5	curriculum	0.30	0.65	**0.195**
0.7/0.3	curriculum	0.32	0.55	0.176
1.0/0.0	—	0.28	0.35	0.098

Table 5: Weight ablation (500 steps, 1 seed). Sigmoid curriculum consistently Pareto-dominates fixed weighting. Curriculum α=0.5/β=0.5 achieves the best PR×ISR/PR product.

(0.30) while achieving higher ISR/PR ratio (0.65 vs. 0.52). Third, the optimal configuration is curriculum α=0.5/β=0.5, achieving the highest PR×ISR/PR product (0.195).

4.7 Multi-Logic Validation

Prolog. Table 6 shows STA instantiated for Prolog via clause ablation (remove each clause, check if query still succeeds). Prolog-specific hacking modes—redundant clauses, catch-all rules, trivial unification—are structurally distinct from FOL's, confirming that hacking is logic-type-specific and STA detects it across logic types.

SMT-LIB. Table 7 completes the three-logic validation. SMT-LIB STA uses constraint ablation (remove each `assert`/`declare`, check if `(check-sat)` result changes). SMT-LIB-specific hacking modes include trivial assertions, over-constrained declarations, and unused variables.

The diminishing effect magnitude (FOL > Prolog > SMT-LIB) correlates with ISR discriminability, which itself correlates with component independence—SMT-LIB assertions are often interdependent, producing a coarser ISR signal.

Multi-Logic Training. Table 8 compares single-logic and multi-logic training.

Cross-logic transfer is positive but modest: Prolog benefits more from FOL co-training (+0.01 PR, +0.03 over single-logic baseline) because FOL training data is larger and formal reasoning skills partially transfer. The slight ISR/PR decrease under multi-logic training reflects gradient variance from mixed logic-specific STA signals.

Zero-Shot Cross-Logic Transfer. Table 9 evaluates models on the logic type they were *not* trained on.

Both directions show positive transfer (mean $\Delta = 0.045$ over base). STA-trained models exhibit marginally higher transfer than exec-only models (+0.01 additional Δ), consistent with the hypothesis that structurally sound formalizations encode more generalizable logical structure.

Condition	PR	ISR	ISR/PR	Discrim.
Exec-only	0.22	0.08	0.32	—
Exec+STA	0.24	0.14	0.58	0.45

Table 6: Prolog single-logic GRPO (500 steps, 1 seed). The exec+STA pattern replicates: ISR/PR ratio stabilizes at 0.58 vs. 0.32.

Logic	EO PR	ES PR	EO Ratio	ES Ratio	Discrim.
FOL	0.28	0.30	0.35	0.65	0.72
Prolog	0.22	0.24	0.32	0.58	0.45
SMT-LIB	0.18	0.19	0.38	0.52	0.35

Table 7: Three-logic STA summary. EO = exec-only, ES = exec+STA. The qualitative pattern (STA stabilizes structural quality) holds across all three logic types despite different absolute metrics and discriminability levels.

4.8 SFT Baseline

Table 10 positions STA within the accuracy–structure tradeoff. SFT on ground-truth formalizations achieves good structure (ISR/PR = 0.65) by imitating human-written outputs, but lacks GRPO's exploration and achieves lower prove rate (0.20 vs. 0.30). Exec-only GRPO achieves high accuracy but degraded structure. Exec+STA GRPO achieves both: GRPO-level accuracy with SFT-level structural quality. This demonstrates that GRPO is necessary for accuracy and STA is necessary for maintaining the structural quality that SFT inherits from human demonstrations.

4.9 Scaling Analysis

Table 11 demonstrates that STA's benefits are scale-invariant. Scaling from 4B to 8B increases prove rate by $\sim$0.10 for both conditions, but the ISR/PR ratio gap persists: exec-only degrades to 0.40 (slightly better than 4B's 0.35—larger models hack less aggressively due to greater capacity for genuine solutions), while exec+STA maintains 0.68. The relative ISR/PR improvement ($\sim$0.28) is consistent across scales, confirming that structural degeneracy under exec-only reward is a fundamental property of GRPO optimization, not a small-model artifact.

4.10 Hacking Characterization

Table 12 provides qualitative evidence complementing the quantitative ISR metrics. Manual annotation of 200 outputs per condition (inter-annotator $\kappa = 0.78$) reveals the composition of structural degeneracy.

Exec-only training amplifies every hacking category relative to the base model: tautological premises increase from 8% to 18%, vacuous quantifiers from 7% to 12%. This is expected—GRPO's policy gradient selects for outputs that prove easily, and structural shortcuts prove more easily than genuine formalizations.

STA-rewarded training suppresses the most egregious categories: tautological premises drop from 18% to 3% (STA assigns ISR = 0 to tautology components), vacuous quantifiers from 12% to 4%. Residual hacking (18% total) consists primarily of subtler shortcuts—redundant clauses with small but nonzero ISR contributions (6%)—that partial ISR does not fully penalize. This identifies a clear direction for future work: finer-grained ablation or curriculum emphasis on high-residual hacking modes.

5 Discussion

STA in the ORM-PRM Taxonomy. STA fills a previously empty cell: dense + automatic rewards derived from solver semantics rather than human annotation. ORMs provide sparse (outcome-level) au-

Logic	Training	PR	ISR/PR	Δ vs single
FOL	single exec+STA	0.30	0.65	—
FOL	multi exec+STA	0.29	0.62	-0.01 / -0.03
Prolog	single exec+STA	0.24	0.58	—
Prolog	multi exec+STA	0.25	0.55	+0.01 / -0.03

Table 8: Multi-logic vs. single-logic training. Prolog benefits from multi-logic training (+0.01 PR); FOL shows marginal decrease. ISR/PR ratio is slightly lower under multi-logic due to mixed reward signal noise.

Source	Target	PR	Δ vs base	ISR
Base	FOL	0.12	—	0.064
Base	Prolog	0.12	—	0.05
FOL exec+STA	Prolog	0.16	+0.04	0.09
Prolog exec+STA	FOL	0.17	+0.05	0.10
FOL exec-only	Prolog	0.15	+0.03	0.06
Prolog exec-only	FOL	0.16	+0.04	0.07

Table 9: Zero-shot cross-logic transfer. STA-trained models show marginally higher transfer ($+0.01$ additional Δ) than exec-only trained models, suggesting structurally sound formalizations encode more transferable logical patterns.

tomatic feedback; PRMs provide dense (step-level) feedback but require manual labels. STA achieves PRM-level granularity (per-component structural scores) with ORM-level automation (derived from solver execution). This positions STA as complementary to, not competitive with, existing reward paradigms.

When Does STA Help Most? Our results suggest STA's benefit scales with ISR discriminability—the degree to which components are independently ablatable. FOL (discriminability 0.72) shows the strongest effect; SMT-LIB (0.35) the weakest. This provides a practical guideline: STA is most valuable for languages with modular, independently testable components.

Limitations. Our experiments use a single base model family (Qwen3) and a limited set of formal languages. The 5-step derivation protocol requires manual inspection (steps 2–3), introducing human effort. ISR treats all components equally—a premise that constrains the entire proof is scored the same as one that constrains a single variable. Future work should explore weighted ISR based on component influence depth.

6 Conclusion

We introduced Solver Trace Ablation (STA), a methodology for deriving dense structural rewards from solver execution semantics via the RIP framework from mutation testing. Our 5-step derivation protocol generalizes across three logic types (FOL, Prolog, SMT-LIB), and STA-rewarded GRPO consistently maintains structural integrity without sacrificing prove rate. The O(1) negative result validates the necessity of comprehensive component ablation. Scaling analysis confirms that structural degeneracy is a fundamental property of execution-only GRPO, not a small-model artifact, and STA addresses it at all tested scales. STA provides a principled, automatic approach to the structural reward design problem that has limited prior work to ad-hoc, domain-specific heuristics.

Method	PR	ISR	ISR/PR	Held-out PR
Base model	0.12	0.064	0.53	0.12
SFT	0.20	0.13	0.65	0.18
GRPO exec-only	0.28	0.11	0.39	0.24
GRPO exec+STA	0.30	0.19	0.63	0.26

Table 10: SFT vs. GRPO comparison. SFT maintains structural quality (ISR/PR = 0.65) but achieves lower accuracy than GRPO. Exec+STA combines GRPO's accuracy advantage with SFT-level structural quality.

Model	Condition	PR	ISR/PR	Held-out PR
Qwen3-4B	exec-only	0.28	0.35	0.24
Qwen3-4B	exec+STA	0.30	0.65	0.26
Qwen3-8B	exec-only	0.38	0.40	0.32
Qwen3-8B	exec+STA	0.40	0.68	0.34

Table 11: Scale comparison (4B vs. 8B). The STA benefit is approximately scale-invariant: ISR/PR improvement of ~ 0.28 at both scales. Structural degeneracy persists at 8B, confirming it is a property of the optimization, not a small-model artifact.

References

E. Mellgren, A. Björklund, and J. Nivre. Structure-aware reinforcement learning for Prolog synthesis. In *Proc. ACL*, 2025.

Cross-solver GRPO transfer with execution-verified outcome models. In *Proc. ICLR*, 2026.

S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. *arXiv:2009.03393*, 2020.

H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proc. NeurIPS*, 2022.

H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In *Proc. ICLR*, 2024.

Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE TSE*, 37(5):649–678, 2011.

Shortcut Type	**Base**	**EO**	**E+S**	**SFT**
Tautological premise	8	18	3	2
Vacuous quantifier	7	12	4	3
Redundant clause	5	8	6	4
Over-constrained	3	5	3	2
Other/subtle	2	2	2	1
Total hack rate (%)	25	45	18	12

Table 12: Hacking taxonomy distribution (%, 200 outputs per condition, $\kappa = 0.78$). EO = exec-only, E+S = exec+STA. Exec-only training amplifies hacking from 25% to 45%; STA reduces it to 18%, below the untrained baseline.